LaaS POC — Updated Runbook

GPU-as-a-Service Infrastructure — As Executed & Validated

March 2026 | Version 2.0 | Status: Phase 2 Complete

> **Purpose of this Document:** This is the definitive as-executed runbook for the LaaS POC. It records exactly what was built, what deviated from the original plan, what was discovered, and what was validated. It serves as the primary handoff document for the software MVP builder who will build the orchestration platform, APIs, and web portal on top of this infrastructure. Every section marked **MVP BUILDER NOTE** contains critical context for building the software layer.

Hardware

POC Test Machine (Single Node)

All POC validation was performed on a single machine. This is NOT one of the 4 production nodes. The GPU architecture differs — POC uses Ada Lovelace (RTX 4090), production will use Blackwell (RTX 5090). Behavioral validation must be re-run on production hardware.

Component	POC Machine (Test)	Production Nodes (x4)
CPU	AMD Ryzen 9 7950X3D (32 threads)	AMD Ryzen 9 9950X3D (32 threads, 3D V-Cache)
GPU	NVIDIA RTX 4090 24GB (Ada, sm_89)	Zotac RTX 5090 32GB (Blackwell, sm_100)
RAM	64GB DDR5	64GB DDR5 G.SKILL 6000MHz
Storage	2TB NVMe	Samsung 990 EVO Plus 2TB NVMe
OS	Ubuntu 22.04 LTS (bare metal)	Ubuntu 22.04 LTS (bare metal on Proxmox host)
Driver	570.211.01	To be installed — same version target
CUDA	12.8.93	12.8 target

Component	POC Machine (Test)	Production Nodes (x4)
Remote	Tailscale VPN (100.100.66.101)	Cloudflare Tunnel (production)

> ⚠ **BLACKWELL NOTE:** RTX 5090 is Blackwell (sm_100). All POC validation was on Ada (sm_89). NVENC concurrent stream limits, MPS fault propagation behavior, and the D3cold reset bug may differ. Explicit re-validation required on production nodes before launch.

Architecture Overview

All user sessions — stateful GUI desktops and ephemeral CLI/Jupyter — run as Selkies EGL Desktop Docker containers. This is the unified container architecture (Architecture A) as defined in the LaaS Infrastructure Plan.

Layer	Technology	Status
Container Runtime	Docker CE 29.3.0	✅ **COMPLETED**
Desktop Streaming	Selkies-GStreamer + WebRTC (ghcr.io image)	✅ **COMPLETED**
Desktop Environment	KDE Plasma 5.27 on Ubuntu 24.04 (inside container)	✅ **COMPLETED**
GPU Sharing — VRAM	HAMi-core (libvgpu.so) — LD_PRELOAD interception	✅ **COMPLETED**
GPU Sharing — SM	CUDA MPS (nvidia-cuda-mps-control daemon)	✅ **COMPLETED**
Video Encoding	NVENC hardware encoding (nvh264enc via GStreamer)	✅ **COMPLETED**
Persistent Storage	ZFS on NAS + NFS mount per user (/home/ubuntu)	✅ **COMPLETED**
Resource Visibility	lxcfs for /proc spoofing (RAM confirmed, CPU partial)	⚠ **DEVIATION**
Process Supervision	supervisord (custom config — critical, see Section 5)	✅ **COMPLETED**
Remote Access	Tailscale (POC) → Cloudflare Tunnel (production)	☐ **PENDING**
Monitoring	Prometheus + Grafana + DCGM Exporter	☐ **PENDING**
Auth	Keycloak	☐ **PENDING**

Disk Layout

POC Machine (Dual-Boot — Different from Production)

The POC machine was already dual-booting Windows 11 and Ubuntu 22.04 when the POC began. The production nodes will be clean installs. The POC layout deviates from the plan.

Partition	Size	Purpose	Note
nvme0n1p1	100MB	/boot/efi	EFI
nvme0n1p2	16MB	MSR (Windows)	DO NOT TOUCH
nvme0n1p3	1.7TB	Windows C:	DO NOT TOUCH
nvme0n1p4	100GB	Ubuntu root (/)	POC only — too small for production
nvme0n1p5	822MB	Recovery	
/vg_containers	—	Directory on root (NOT dedicated partition)	POC workaround

Production Node Target Layout (per node, 2TB)

```
nvme0n1p1   128GB   EXT4    →  OS + swap
nvme0n1p2   500GB   EXT4    →  /var/lib/docker  (images, overlays)
nvme0n1p3  1100GB   LVM PV →  VG_CONTAINERS (ephemeral scratch)
nvme0n1p4   240GB   EXT4    →  Shared read-only datasets
```

> **MVP BUILDER NOTE:** On production nodes, Docker storage is on a dedicated 500GB partition, not the OS partition. Container writable overlays are ephemeral — destroyed on container stop. User persistent data lives entirely on NAS via NFS. The orchestrator never needs to manage local disk for user data.

Pre-Flight Setup

Step 0A — BIOS Configuration

Required settings verified before POC:

- AMD-Vi (IOMMU): Enabled
- SVM Mode (AMD-V): Enabled
- Resizable BAR / SAM: Enabled
- XMP/EXPO: Enabled (RAM stability)
- BIOS version 1710 → updated via ASUS EZ Flash 3 before POC

Step 0B — NVIDIA Driver + CUDA

```
# Verify (already installed on POC machine):
nvidia-smi
# Expected: RTX 4090, Driver 570.211.01
nvcc --version
# Expected: CUDA 12.8.93

# If installing fresh:
sudo add-apt-repository ppa:graphics-drivers/ppa -y && sudo apt update
sudo apt install -y nvidia-driver-570 nvidia-utils-570
sudo reboot

# CUDA Toolkit 12.8:
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update && sudo apt install -y cuda-toolkit-12-8
```

Step 0C — Docker CE + NVIDIA Container Toolkit

```
curl -fsSL https://get.docker.com | sudo bash
sudo usermod -aG docker $USER && newgrp docker

# NVIDIA Container Toolkit:
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
  sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list | \
  sed 's#deb https://#deb [signed-by=...keyring.gpg] https://#g' | \
  sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
sudo apt update && sudo apt install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

> ⚠ **DEVIATION:** The original plan used '--gpus all' for GPU passthrough. In practice, --device flags are more reliable with custom images. Both methods work. Use --gpus all with standard images, --device flags with cuda-hami-env.
>
> ⚠ **DEVIATION:** nvidia/cuda:12.8.0-base and runtime images do NOT include nvidia-smi binary. To test GPU in a clean container, mount nvidia-smi and libnvidia-ml.so.1 from the host into ubuntu:22.04 instead.

HAMi-Core Build and Validation

What HAMi-Core Does

HAMi-core provides VRAM and SM (compute) limits per container via LD_PRELOAD interception of CUDA API calls. It is NOT a kernel module or daemon — it is a shared library (libvgpu.so) that intercepts cuMemAlloc, cuMemGetInfo, and related calls.

Key mechanism: Each container that loads libvgpu.so via LD_PRELOAD creates or attaches to a file-backed shared memory region at `/tmp/cudevshr.cache`. This file stores the memory limits and per-process usage tracking. Limits are read from environment variables at init time.

Step 1 — Build libvgpu.so

```
sudo apt install -y build-essential cmake git
git clone https://github.com/Project-HAMi/HAMi-core.git
cd HAMi-core && mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
ls -la libvgpu.so   # Confirm: ~418784 bytes

# CORRECT install path (DEVIATION from original plan):
sudo cp libvgpu.so /usr/lib/libvgpu.so   # NOT /usr/local/lib/
sudo ldconfig
ls -la /usr/lib/libvgpu.so
```

> ⚠ **CRITICAL DEVIATION:** libvgpu.so must be at /usr/lib/libvgpu.so — NOT /usr/local/lib/. This is the correct path for bare metal Ubuntu. Using the wrong path causes silent failures where HAMi initializes but LD_PRELOAD is never resolved.

Step 2 — CUDA MPS Daemon

```
# Stop any manually-started MPS first:
echo 'quit' | sudo nvidia-cuda-mps-control 2>/dev/null || true

sudo tee /etc/systemd/system/cuda-mps.service > /dev/null << 'EOF'
[Unit]
Description=CUDA MPS Control Daemon
After=nvidia-persistenced.service
[Service]
Type=forking
Environment=CUDA_VISIBLE_DEVICES=0
Environment=CUDA_MPS_PIPE_DIRECTORY=/tmp/nvidia-mps
Environment=CUDA_MPS_LOG_DIRECTORY=/tmp/nvidia-log
ExecStartPre=/usr/bin/nvidia-smi -pm 1
ExecStart=/usr/bin/nvidia-cuda-mps-control -d
ExecStop=/bin/bash -c "echo quit | /usr/bin/nvidia-cuda-mps-control"
Restart=on-failure
RestartSec=10
[Install]
```

```
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now cuda-mps
sudo systemctl status cuda-mps   # Expect: active (running)
```

> ⚠ **NOTE:** MPS get_server_list returning empty is NORMAL. Servers spawn on-demand when a CUDA process connects. This is not a failure.

Step 3 — HAMi Enforcement Validation

This is the definitive test. Run inside a container with LD_PRELOAD set:

```
# Install torch inside container:
pip install torch --index-url https://download.pytorch.org/whl/cu128 -q --break-system-packages

# Run enforcement test:
python3 -c "
import torch
total = torch.cuda.get_device_properties(0).total_memory
print(f'Reported VRAM: {total/1024**3:.1f}GB')   # Should show 4.0GB

print('Trying to allocate 6GB (should FAIL if limit is 4GB)...')
try:
    t = torch.zeros(int(6*1024**3/4), dtype=torch.float32, device='cuda')
    print('FAIL: limit NOT enforced')
except RuntimeError as e:
    print(f'SUCCESS: blocked — {e}')

print('Trying to allocate 3GB (should SUCCEED)...')
try:
    t = torch.zeros(int(3*1024**3/4), dtype=torch.float32, device='cuda')
    print('SUCCESS: 3GB allocated fine')
except RuntimeError as e:
    print(f'FAIL: blocked at 3GB — unexpected')
"
```

Test	Expected	Actual Result	Status
Reported VRAM (torch)	4.0 GB	4.0 GB	✅**PASSED**
6GB allocation attempt	BLOCKED (OOM)	BLOCKED — HAMi OOM fired at 4294967296 bytes	✅**PASSED**
3GB allocation attempt	SUCCEEDS	Allocated fine	✅**PASSED**

Test	Expected	Actual Result	Status
nvidia-smi VRAM display	Shows full 24GB (expected — NVML bypass)	Shows 24GB	✅PASSED
SM limit (25%)	Enforced via MPS ACTIVE_THREAD_PERCENTAGE	Set in cudevshr.cache	✅PASSED

> **IMPORTANT — nvidia-smi always shows full GPU:** nvidia-smi queries NVML directly at the kernel driver level. HAMi cannot intercept NVML calls — only CUDA API calls. This means nvidia-smi will always report the full GPU VRAM regardless of limits. This is expected behavior, not a bug. The actual enforcement (HAMi OOM) is real and works correctly. The recommended approach: wrap the nvidia-smi binary (see Section 7) and document this limitation in Terms of Service.

Technical detail — how HAMi stores limits: When a CUDA process with LD_PRELOAD loads, HAMi creates or attaches to `/tmp/cudevshr.cache` — a file-backed mmap'd shared region (struct shared_region_t, ~2MB). The limit[] array at offset ~1600 bytes holds per-device VRAM limits in bytes. The first process to create the file writes the limits from env vars. Subsequent processes in the same container attach to the same file and verify consistency. Each container gets its own isolated cache file (different Docker overlay filesystem). This is per-container and correct.

Selkies EGL Desktop Image

Image Details

Property	Value
Image	ghcr.io/selkies-project/nvidia-egl-desktop:latest
Base OS inside container	Ubuntu 24.04 (NOT 22.04 — important for software installs)
Desktop	KDE Plasma 5.27.12
Default user	ubuntu (uid 1000)
Default password	Set via PASSWD env var
Streaming	selkies-gstreamer (WebRTC, GStreamer pipeline)
Video encoder	nvh264enc (NVIDIA NVENC hardware encoding)
Port (internal)	Set via SELKIES_PORT env var (NOT fixed 8080)
Web proxy	nginx (listens on NGINX_PORT env var)

> ⚠ **DEVIATION:** The plan called for building the Selkies image from source (20-40 min). In practice, the pre-built ghcr.io image was used — faster and more reliable. Build from source only when custom software needs to be baked in. The image uses Ubuntu 24.04 internally, not 22.04 as originally planned.

Pulling the Image

```
docker pull ghcr.io/selkies-project/nvidia-egl-desktop:latest
docker images | grep selkies

# Pre-pull on ALL nodes before launch (production):
# This eliminates pull time from session startup latency
```

> **MVP BUILDER NOTE:** The image must be pre-pulled on all 4 production nodes. Build a startup check in the orchestrator that verifies image availability on each node at boot. Do not launch containers on a node where the image is not yet pulled.

Critical: supervisord Configuration

Why This Configuration is Non-Negotiable

The Selkies container has multiple process trees that must NOT all have LD_PRELOAD set. VGL (VirtualGL) and the X server/KDE startup will crash or produce rendering errors if libvgpu.so is loaded globally. The supervisord config below is the result of extensive debugging and is the validated working solution.

Process	LD_PRELOAD	Why
entrypoint (X server, VGL, KDE)	NONE	VGL dlopen fails if libvgpu.so is in LD_PRELOAD
selkies-gstreamer (NVENC encoder)	INLINE in command only	Needs CUDA interception for encoder VRAM accounting
User Konsole/terminal sessions	Via /etc/bash.bashrc	Users need HAMi limits when running CUDA workloads
nvidia-smi binary	WRAPPED (see Section 7)	Prevents segfault from NVML + HAMi interaction

supervisord-hami.conf — Complete Working Config

Save this to /tmp/supervisord-hami.conf on the host and mount into every container:

```
cat > /tmp/supervisord-hami.conf << 'SUPERVISORD_EOF'
[unix_http_server]
file=/tmp/supervisor.sock
chmod=0700

[supervisord]
logfile=/tmp/supervisord.log
logfile_maxbytes=5MB
```

```
logfile_backups=0
loglevel=info
pidfile=/tmp/supervisord.pid
childlogdir=/tmp
nodaemon=true

[rpcinterface:supervisor]
supervisor.rpcinterface_factory = supervisor.rpcinterface:make_main_rpcinterface

[supervisorctl]
serverurl=unix:///tmp/supervisor.sock

[include]
files = /etc/supervisor/conf.d/*.conf

[program:entrypoint]
command=bash -c "dbus-run-session -- /etc/entrypoint.sh"
stdout_logfile=/tmp/entrypoint.log
stdout_logfile_maxbytes=5MB
redirect_stderr=true
stopasgroup=true
stopsignal=INT
autostart=true
autorestart=true
priority=1

[program:dbus]
command=bash -c "mkdir -pm700 \"${XDG_RUNTIME_DIR}\"; ...; dbus-daemon --system ..."
environment=DISPLAY=%(ENV_DISPLAY)s,...
# (see full config in repo)

[program:selkies-gstreamer]
# CRITICAL: LD_PRELOAD is set INLINE here, NOT in docker -e LD_PRELOAD
command=bash -c "if [ \"$(echo ${KASMVNC_ENABLE}...)\" != \"true\" ]; then \
  LD_PRELOAD=/usr/lib/libvgpu.so /etc/selkies-gstreamer-entrypoint.sh; \
  else sleep infinity; fi"
stdout_logfile=/tmp/selkies-gstreamer-entrypoint.log
stdout_logfile_maxbytes=5MB
redirect_stderr=true
stopasgroup=true
stopsignal=INT
autostart=true
autorestart=true
priority=20

# ... nginx, pipewire, kasmvnc programs follow
SUPERVISORD_EOF
```

bash.bashrc Injection for User Shell Sessions

User terminal sessions (Konsole inside the desktop) are non-login shells. /etc/profile.d/ scripts are NOT sourced. LD_PRELOAD must be injected via /etc/bash.bashrc:

```
# Get original bashrc from image
docker run --rm --entrypoint cat \
  ghcr.io/selkies-project/nvidia-egl-desktop:latest \
  /etc/bash.bashrc > /tmp/bash.bashrc.orig

# Append HAMi injection
cat >> /tmp/bash.bashrc.orig << 'EOF'

# HAMi CUDA interception — applied to all bash sessions
# NOT set globally — only activates when user opens a terminal
# VGL/X/KDE startup is unaffected (uses /etc/entrypoint.sh, not login shells)
if [ -f /usr/lib/libvgpu.so ] && [ "${HAMI_INJECTED}" != "1" ]; then
  export LD_PRELOAD=/usr/lib/libvgpu.so
  export HAMI_INJECTED=1
  mkdir -p /tmp/vgpulock 2>/dev/null
fi
EOF
chmod 644 /tmp/bash.bashrc.orig
```

> ⚠ **DEVIATION:** profile.d scripts are NOT sourced for non-login shells. Konsole opens non-login interactive shells. /etc/bash.bashrc is the correct injection point. This was discovered after multiple failed attempts with profile.d.

Binary Wrappers

nvidia-smi Wrapper

nvidia-smi segfaults when both libvgpu.so LD_PRELOAD and NVML are active simultaneously. Wrap it to strip LD_PRELOAD before execution:

```
cat > /tmp/nvidia-smi-wrapper << 'EOF'
#!/bin/bash
# Strip HAMi LD_PRELOAD before calling real nvidia-smi
exec env -u LD_PRELOAD /usr/bin/nvidia-smi.real "$@"
EOF
chmod +x /tmp/nvidia-smi-wrapper
```

> □ **MVP BUILDER NOTE:** nvidia-smi will always show the full host GPU VRAM (e.g., 24GB for RTX 4090, 32GB for RTX 5090) regardless of container limits. This is a kernel driver limitation — NVML bypasses all user-space interception. The wrapper prevents segfaults. Do NOT attempt to spoof nvidia-smi output — focus on real enforcement (HAMi) and document the limitation transparently.

passwd Wrapper

```
cat > /tmp/passwd-wrapper << 'EOF'
#!/bin/bash
# Strip LD_PRELOAD before calling real passwd (prevents segfault)
exec env -u LD_PRELOAD /usr/bin/passwd.real "$@"
EOF
chmod +x /tmp/passwd-wrapper
```

NFS Persistent Storage Setup

Architecture

Each user gets exactly 15GB of persistent storage as a ZFS dataset on the NAS, mounted via NFS to whichever node their session runs on. This is the source of truth for user data across all sessions.

Property	Value
NAS filesystem	ZFS (TrueNAS Scale or zfsutils-linux)
Pool	datapool
User dataset path	datapool/users/<uid>
Quota	15GB hard limit enforced at ZFS level
Mount point in container	/home/ubuntu
NFS protocol	NFSv4 over 10GbE
POC simulation	File-backed ZFS pool (20GB), 5GB quota per user (space constraint)

POC Setup Commands

```
sudo apt install -y zfsutils-linux nfs-kernel-server

# POC: file-backed pool (no dedicated partition available)
sudo mkdir -p /vg_containers
sudo truncate -s 20G /vg_containers/nas_pool.img
sudo zpool create -f datapool /vg_containers/nas_pool.img

# Create user volumes
sudo zfs create datapool/users
sudo zfs create -o quota=5G datapool/users/testuser1
sudo zfs create -o quota=5G datapool/users/testuser2
```

```
# NFS export (loopback for POC)
echo '/datapool/users 127.0.0.1(rw,sync,no_subtree_check,no_root_squash)' | sudo tee -a /etc/exports
sudo exportfs -ra
sudo systemctl restart nfs-kernel-server

# Mount
sudo mkdir -p /mnt/nfs/users
sudo mount -t nfs4 127.0.0.1:/datapool/users /mnt/nfs/users

# Create home dirs with correct ownership (uid 1000 = ubuntu inside container)
for i in 1 2 3 4; do
  sudo mkdir -p /mnt/nfs/users/testuser$i
  sudo chown 1000:1000 /mnt/nfs/users/testuser$i
done
```

> ⚠ **DEVIATION:** POC uses 20GB file-backed pool with 5GB quotas (not 15GB) due to root partition space constraints. Production uses dedicated NAS hardware with 15GB quotas per user as specified in the infra plan.

> **MVP BUILDER NOTE:** The NFS path exposed inside containers is 127.0.0.1:/datapool/users/testuser1 — this leaks internal infrastructure details. In production, use a resolvable internal hostname (e.g., nas.internal) instead of 127.0.0.1, and use abstract user IDs as path components instead of usernames (e.g., /homes/u_a7f3c2). The orchestrator must provision a new ZFS dataset and NFS export when a new user registers, and mount it to /home/ubuntu in every session that user launches.

Resource Visibility: lxcfs

Problem

Without intervention, containers show the host's full hardware specs in /proc and KDE System Settings. A user on a 4-core/8GB tier sees the full 32-core/64GB host machine. This exposes internal infrastructure and is a trust/security issue.

Solution: lxcfs

lxcfs is a FUSE filesystem that intercepts /proc reads and returns container-aware values matching Docker cgroup limits.

```
# Install and start on HOST (not inside containers):
sudo apt install -y lxcfs
sudo systemctl enable --now lxcfs

# Verify it provides all needed files:
ls /var/lib/lxcfs/proc/
```

```
# Expected: cpuinfo  diskstats  loadavg  meminfo  slabinfo  stat  swaps  uptime
```

Resource	Without lxcfs	With lxcfs	Status
RAM (free -h)	Shows full host 62GB	Shows container limit (8GB)	✅ **COMPLETED**
CPU count (nproc)	Shows host 32 cores	Needs --cpuset-cpus flag too	⚠ **DEVIATION**
KDE System Info RAM	Shows host 62GB	Shows container 8GB	✅ **COMPLETED**
KDE System Info CPU	Shows host 32 cores	Partial fix (see below)	⚠ **DEVIATION**
df -h	Shows real host partitions	Not fixable via lxcfs (kernel mounts)	□ **FOR MVP**
nvidia-smi VRAM	Shows full GPU always	Not fixable (NVML kernel bypass)	□ **FOR MVP**

> **CPU NOTE:** lxcfs alone does not fix nproc. --cpus=4 limits CPU time (throttling) but all 32 cores remain visible. Add --cpuset-cpus=0-3 to docker run to restrict visibility to 4 physical cores. lxcfs then correctly reports 4 in /proc/cpuinfo.

> □ **MVP BUILDER NOTE:** df -h will always show the container overlay device and NFS mount path. This is a kernel-level reality — containers see their actual mounts. Mitigation for production: (1) Use abstract NFS paths with no identifying information. (2) The overlay root line (Device: overlay) is normal and reveals nothing sensitive. (3) Do not expose an SSH shell to users — the desktop-only UX limits what df output can expose.

Validated Container Launch Parameters

Complete Working docker run Command

This is the exact validated command for launching one Selkies desktop session. All deviations and required flags are documented inline:

```
# Prepare per-container vgpulock dir (MUST be per-container, not shared):
sudo mkdir -p /tmp/vgpulock-$i && sudo chmod 777 /tmp/vgpulock-$i

docker run -d \
  --name selkies-$SESSION_ID \

  # Resource limits (adjust per tier):
  --gpus all                          # GPU access
  --cpus=4                            # CPU time limit
  --cpuset-cpus=0-3                   # CPU visibility (lxcfs needs this for nproc)
  --memory=8g                         # RAM hard limit
```

```
  # IPC and networking:
  --ipc=host                          # Required for Selkies shared memory (WebRTC)
  --network=host                      # POC ONLY — required for WebRTC over Tailscale
  --tmpfs /dev/shm:rw                 # Shared memory for WebRTC

  # Display and streaming:
  -e DISPLAY=:$DISPLAY_NUM            # Unique per container (e.g., :20, :21, :22,
:23)
  -e DISPLAY_SIZEW=1920 -e DISPLAY_SIZEH=1080 -e DISPLAY_REFRESH=60 \
  -e SELKIES_ENCODER=nvh264enc        # Hardware NVENC encoding

  # Auth:
  -e SELKIES_ENABLE_BASIC_AUTH=true \
  -e SELKIES_BASIC_AUTH_PASSWORD=$TOKEN \  # Per-session random token
  -e PASSWD=mypasswd                  # Desktop login password

  # Port assignment (unique per container — --network=host requires this):
  -e NGINX_PORT=$NGINX_P              # Web UI port (e.g., 8081, 8082, 8083, 8084)
  -e SELKIES_PORT=$SELKIES_P          # Internal gstreamer port (e.g., 9081, 9082)
  -e SELKIES_METRICS_HTTP_PORT=$METRICS_P \

  # GPU:
  -e CUDA_VISIBLE_DEVICES=0           # GPU index
  -e CUDA_NVRTC_ARCH=89               # sm_89 for Ada (sm_100 for Blackwell RTX 5090)
  -e __NV_PRIME_RENDER_OFFLOAD=1 \
  -e __GLX_VENDOR_LIBRARY_NAME=nvidia \

  # HAMi VRAM limits (adjust per tier):
  -e CUDA_DEVICE_MEMORY_LIMIT_0=4096m    # 4GB VRAM limit for this container
  -e CUDA_DEVICE_SM_LIMIT=25             # 25% SM (compute) limit
  -e CUDA_MPS_PINNED_DEVICE_MEM_LIMIT='0=4G' \
  -e CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=25 \

  # TURN server for WebRTC (required for remote access through NAT/firewall):
  -e SELKIES_TURN_HOST=$TURN_HOST \
  -e SELKIES_TURN_PORT=3478 \
  -e SELKIES_TURN_USERNAME=$TURN_USER \
  -e SELKIES_TURN_PASSWORD=$TURN_PASS \
  -e SELKIES_TURN_PROTOCOL=tcp \

  # Volume mounts:
  -v /mnt/nfs/users/$USER_ID:/home/ubuntu \  # User persistent storage
  -v /usr/lib/libvgpu.so:/usr/lib/libvgpu.so \  # HAMi library
  -v /tmp/vgpulock-$i:/tmp/vgpulock \          # Per-container HAMi lock dir
  -v /usr/bin/nvidia-smi:/usr/bin/nvidia-smi.real \  # Real nvidia-smi
  -v /tmp/nvidia-smi-wrapper:/usr/bin/nvidia-smi \   # Wrapper replaces it
  -v /usr/bin/passwd:/usr/bin/passwd.real \
```

```
  -v /tmp/passwd-wrapper:/usr/bin/passwd \
  -v /tmp/supervisord-hami.conf:/etc/supervisord.conf \  # Critical custom config
  -v /tmp/bash.bashrc.orig:/etc/bash.bashrc \           # HAMi injection for terminals

  # lxcfs /proc spoofing:
  -v /var/lib/lxcfs/proc/cpuinfo:/proc/cpuinfo:ro \
  -v /var/lib/lxcfs/proc/meminfo:/proc/meminfo:ro \
  -v /var/lib/lxcfs/proc/stat:/proc/stat:ro \
  -v /var/lib/lxcfs/proc/uptime:/proc/uptime:ro \
  -v /var/lib/lxcfs/proc/loadavg:/proc/loadavg:ro \
  -v /var/lib/lxcfs/proc/diskstats:/proc/diskstats:ro \
  -v /var/lib/lxcfs/proc/swaps:/proc/swaps:ro \

  ghcr.io/selkies-project/nvidia-egl-desktop:latest
```

> ⚠ **PRODUCTION NETWORK CHANGE:** --network=host is used in the POC because Tailscale VPN is the remote access method and WebRTC requires direct port access. In production, Cloudflare Tunnel handles remote access — it terminates at the host and reverse-proxies to containers. This means --network=bridge can and SHOULD be used in production for proper network namespace isolation between containers. With --network=bridge, use -p $NGINX_P:8080 and configure the Cloudflare Tunnel to proxy to the container's exposed port.

Tier Resource Configurations

> **MVP BUILDER NOTE:** The orchestrator must map each user's subscription tier to a specific set of resource parameters at container launch time. The table below defines the validated parameter sets for each tier. These are passed as environment variables and Docker flags to docker run.

Tier	CPUs	RAM	VRAM	SM%	GPU Access	Session Type
Starter	2	4GB	None	None	No	Stateful GUI
Standard	4	8GB	None	None	No	Stateful GUI
Pro	4	8GB	4GB	25%	Yes (shared)	Stateful GUI
Power	6	16GB	8GB	35%	Yes (shared)	Stateful GUI
Max	8	24GB	16GB	50%	Yes (shared)	Stateful GUI
Full Machine	16	64GB	32GB	100%	Yes (exclusive)	Stateful GUI
Ephemeral CPU	2-4	4-8GB	None	None	No	CLI/Jupyter
Ephemeral GPU	4	8GB	4-8GB	25-35%	Yes (shared)	CLI/Jupyter

HAMi env vars per tier: CUDA_DEVICE_MEMORY_LIMIT_0 (e.g., 4096m, 8192m, 16384m, 32768m), CUDA_DEVICE_SM_LIMIT (e.g., 25, 35, 50, 100), CUDA_MPS_PINNED_DEVICE_MEM_LIMIT (e.g., '0=4G'), CUDA_MPS_ACTIVE_THREAD_PERCENTAGE (same as SM_LIMIT).

Multi-Container Port Assignment

The Port Problem and Solution

With --network=host, all containers share the host network namespace. Multiple containers cannot all bind to port 8080. Each container must be assigned unique ports via environment variables.

Container Slot	NGINX_PORT (Web UI)	SELKIES_PORT (Internal)	METRICS_PORT	DISPLAY
1	8081	9081	19081	:20
2	8082	9082	19082	:21
3	8083	9083	19083	:22
4	8084	9084	19084	:23
N (general)	8080+N	9080+N	19080+N	:19+N

> ⚠ **DEVIATION:** The original plan used -p 8080:8080 port mapping. This causes 'Connection failed' in WebRTC when using --network=host. With --network=host, -p flags are ignored. Use SELKIES_PORT and NGINX_PORT env vars only. In production with --network=bridge, revert to -p $NGINX_P:$INTERNAL_PORT.

> **MVP BUILDER NOTE:** The orchestrator port registry: maintain a mapping of (node_id, slot_number) → port assignments. When launching a container, assign the next available slot on the target node. On container stop, free the slot. With --network=bridge in production, port conflicts are eliminated — each container has its own network namespace.

Session Lifecycle for Orchestrator

> **MVP BUILDER NOTE:** This section defines exactly what the orchestrator must do at each stage of a user session. These are the infrastructure operations — the software layer calls these.

Session Launch Sequence

1. Check resource availability on target node (VRAM budget, CPU slots, RAM headroom)

2. Provision NFS home dir if first session for this user: zfs create -o quota=15G datapool/users/$uid
3. Create per-container vgpulock dir: mkdir -p /tmp/vgpulock-$session_id && chmod 777
4. Generate session token: TOKEN=$(openssl rand -hex 16)
5. Assign port slot (NGINX_PORT, SELKIES_PORT, METRICS_PORT, DISPLAY number)
6. Execute docker run with all parameters from Section 9
7. Poll container health: wait for nginx to listen on NGINX_PORT (check every 2s, timeout 60s)
8. Return session URL and token to frontend when ready (~15-25 seconds)

Session Health Check

```
# Check if nginx is up (container ready for connection):
ss -tlnp | grep :$NGINX_PORT

# Or via docker inspect:
docker inspect --format='{{.State.Status}}' selkies-$SESSION_ID

# Check selkies-gstreamer is running (streaming ready):
docker exec selkies-$SESSION_ID grep 'RUNNING' /tmp/selkies-gstreamer-entrypoint.log |
tail -1
```

Session Termination

9. Send graceful warning to user (15-min, 5-min before scheduled end)
10. docker stop selkies-$SESSION_ID (SIGTERM → graceful KDE shutdown → 30s timeout → SIGKILL)
11. docker rm selkies-$SESSION_ID (destroys writable overlay — all system-level changes lost)
12. User NFS home dir is preserved (persists indefinitely)
13. Free port slot in orchestrator registry
14. Free VRAM budget on node
15. Record session end, duration, resource usage for billing

Session Reconnection

Container continues running if browser closes mid-session (until scheduled end time). The orchestrator returns the same NGINX_PORT and token to the user when they reconnect — WebRTC re-establishes automatically.

Container Startup Time

Current measured cold-start time: ~15-25 seconds to KDE Plasma fully visible in browser. Primary bottleneck is KDE Plasma initialization. Future optimization: pre-warm idle containers per node so users get instant sessions.

Monitoring Stack (Bonus — Partially Implemented)

```
cat > prometheus.yml << 'EOF'
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: 'dcgm'
    static_configs:
      - targets: ['localhost:9400']
EOF

cat > docker-compose-monitoring.yml << 'EOF'
version: '3.8'
services:
  dcgm-exporter:
    image: nvcr.io/nvidia/k8s/dcgm-exporter:latest
    runtime: nvidia
    environment:
      NVIDIA_VISIBLE_DEVICES: all
    ports: ['9400:9400']
    restart: always
  prometheus:
    image: prom/prometheus:latest
    volumes: ['./prometheus.yml:/etc/prometheus/prometheus.yml']
    ports: ['9090:9090']
    restart: always
  grafana:
    image: grafana/grafana:latest
    ports: ['3000:3000']
    environment:
      GF_SECURITY_ADMIN_PASSWORD: admin
    restart: always
EOF

docker compose -f docker-compose-monitoring.yml up -d
```

Phase 2 — Final Validation Checklist (Step 2.13)

Component	Pass Condition	Status	Notes
GPU Driver	RTX 4090, 570.x driver	✅**PASSED**	570.211.01

Component	Pass Condition	Status	Notes
CUDA	nvcc shows 12.8.x	✅PASSED	12.8.93
Docker GPU	Container sees GPU	✅PASSED	24564MiB
HAMi-core build	libvgpu.so at /usr/lib/	✅PASSED	418784 bytes
HAMi VRAM reporting	torch reports limited VRAM	✅PASSED	4.0GB reported (4GB tier)
HAMi VRAM enforcement	OOM at limit	✅PASSED	6GB blocked, 3GB allowed
HAMi escape resistance	Single-shot + chunk allocation blocked	✅PASSED	
HAMi stateless isolation	No state bleed between containers	✅PASSED	Per-container cudevshr.cache
MPS Daemon	systemctl shows active/running	✅PASSED	
Selkies Image	ghcr.io image pulled	✅PASSED	Ubuntu 24.04 base
Single Selkies Session	Browser connects, KDE Plasma visible	✅PASSED	
GPU in Selkies	CUDA works inside desktop	✅PASSED	
NVENC Encoding	nvh264enc active in logs	✅PASSED	
supervisord HAMi config	VGL clean, HAMi in gstreamer, terminals	✅PASSED	No VGL dlopen errors
NFS Home Dir	Files persist after container stop	✅PASSED	ZFS dataset on loopback pool
lxcfs RAM spoofing	/proc/meminfo shows container limit	✅PASSED	8GB shown correctly
lxcfs CPU spoofing	nproc shows container CPU count	⚠ DEVIATION	Needs --cpuset-cpus flag added
4 Concurrent Sessions	All 4 accessible with correct limits	☐ PENDING	
Container Fault Isolation	Container 1 crash, others survive	☐ PENDING	
Container Restart	NFS home intact after restart	☐ PENDING	

Known Limitations and Production Considerations

What Requires Production Validation

- RTX 5090 (Blackwell sm_100) — all tests were on RTX 4090 (Ada sm_89). NVENC concurrent limits, HAMi behavior, MPS fault propagation must be re-validated.
- CUDA_NVRTC_ARCH must change from 89 to 100 for Blackwell in all container launch commands.
- The D3cold GPU reset bug (Proxmox forum, NVIDIA forums) — RTX 5090 specific. Must test VM passthrough stability if Architecture B is ever used.

Security Gaps to Address Before Production

- --network=host in POC: Switch to --network=bridge in production with Cloudflare Tunnel. This is the most critical security fix — --network=host means all containers share the host network namespace.
- --ipc=host: Required for Selkies WebRTC shared memory. Evaluate whether --ipc=private breaks WebRTC in production environment.
- df -h leaks host partition info: Use abstract NFS paths (hostname not IP, obfuscated user IDs).
- nvidia-smi always shows full GPU: Document in ToS. Cannot be fixed at kernel level.
- Kata Containers (optional upgrade path): Provides genuine VM-level isolation (own kernel, own /proc) while keeping Docker API. Requires KVM validation on Proxmox nodes. Adds ~2-3 seconds to startup. Investigate for v2.

MVP Software Builder: Key Facts Summary

If you are the engineer building the orchestration platform, APIs, and web portal on top of this infrastructure, here is everything you need to know:

What You Need to Know	Details
How to launch a session	docker run with the exact parameters in Section 9. No Kubernetes, no Swarm — direct Docker API.
How to check if session is ready	Poll for nginx listening on NGINX_PORT. Typically ready in 15-25 seconds.
How to stop a session	docker stop → docker rm. NFS home persists. Everything else is ephemeral.
How to provision user storage	zfs create -o quota=15G datapool/users/$uid on NAS. Mount via NFS at container launch.
How VRAM limits work	env vars at docker run time → HAMi reads them at first CUDA call → writes to /tmp/cudevshr.cache. Limits are real and enforced.
Why nvidia-smi shows full GPU	NVML kernel bypass. Cannot be fixed. Document in ToS. Real enforcement is HAMi.

What You Need to Know	Details
Port assignment	NGINX_PORT=8080+slot, SELKIES_PORT=9080+slot, DISPLAY=:19+slot. Unique per container on same node.
Multi-node session routing	NFS home follows user to any node. Orchestrator picks node with available VRAM budget. Return correct host IP + port to frontend.
lxcfs requirement	Must be installed and running on every host node. Required for RAM/CPU visibility in containers.
Pre-built files needed on host	supervisord-hami.conf, bash.bashrc.orig, nvidia-smi-wrapper, passwd-wrapper, libvgpu.so. These must be deployed to /tmp/ on each node before any sessions launch.
Session startup latency	~15-25 seconds cold start. Pre-warming pool recommended for production UX.
The Selkies image OS	Ubuntu 24.04 internally (not 22.04). Software installs inside container use apt for Ubuntu 24.04 packages.

End of Document — LaaS POC Runbook v2.0

Learnings / Notes

Working docker command! (reference)

```
docker stop selkies-test-1 && docker rm selkies-test-1


TOKEN=$(openssl rand -hex 16)
SESSION_HOST="ws-$(openssl rand -hex 4)"
echo "Access: http://100.100.66.101:8081 | token: $TOKEN | hostname: $SESSION_HOST"


docker run -d \
  --name selkies-test-1 \
  --hostname $SESSION_HOST \
  --gpus all \
  --cpus=4 \
  --cpuset-cpus=0-3 \
```

```
--memory=8g \
--ipc=host --network=host --tmpfs /dev/shm:rw \
-e TZ=UTC \
-e DISPLAY=:20 \
-e DISPLAY_SIZEW=1920 -e DISPLAY_SIZEH=1080 -e DISPLAY_REFRESH=60 \
-e SELKIES_ENCODER=nvh264enc \
-e SELKIES_ENABLE_BASIC_AUTH=true \
-e SELKIES_BASIC_AUTH_PASSWORD=$TOKEN \
-e NGINX_PORT=8081 \
-e SELKIES_PORT=9081 \
-e SELKIES_METRICS_HTTP_PORT=19081 \
-e PASSWD=mypasswd \
-e CUDA_VISIBLE_DEVICES=0 \
-e CUDA_NVRTC_ARCH=89 \
-e __NV_PRIME_RENDER_OFFLOAD=1 \
-e __GLX_VENDOR_LIBRARY_NAME=nvidia \
-e CUDA_DEVICE_MEMORY_LIMIT_0=4096m \
-e CUDA_DEVICE_SM_LIMIT=25 \
-e CUDA_MPS_PINNED_DEVICE_MEM_LIMIT="0=4G" \
-e CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=25 \
-e SELKIES_TURN_HOST=106.51.247.170 \
-e SELKIES_TURN_PORT=3478 \
-e SELKIES_TURN_USERNAME=selkies \
-e SELKIES_TURN_PASSWORD=wVIAbfwkgkxjaCiZVX4BDsdU \
-e SELKIES_TURN_PROTOCOL=tcp \
-v /mnt/nfs/users/testuser1:/home/ubuntu \
-v /usr/lib/libvgpu.so:/usr/lib/libvgpu.so \
-v /tmp/vgpulock-1:/tmp/vgpulock \
-v /usr/bin/nvidia-smi:/usr/bin/nvidia-smi.real \
```

```
  -v /tmp/nvidia-smi-wrapper:/usr/bin/nvidia-smi \
  -v /usr/bin/passwd:/usr/bin/passwd.real \
  -v /tmp/passwd-wrapper:/usr/bin/passwd \
  -v /tmp/supervisord-hami.conf:/etc/supervisord.conf \
  -v /tmp/bash.bashrc.orig:/etc/bash.bashrc \
  -v /var/lib/lxcfs/proc/cpuinfo:/proc/cpuinfo:ro \
  -v /var/lib/lxcfs/proc/meminfo:/proc/meminfo:ro \
  -v /var/lib/lxcfs/proc/stat:/proc/stat:ro \
  -v /var/lib/lxcfs/proc/uptime:/proc/uptime:ro \
  -v /var/lib/lxcfs/proc/loadavg:/proc/loadavg:ro \
  -v /var/lib/lxcfs/proc/diskstats:/proc/diskstats:ro \
  -v /var/lib/lxcfs/proc/swaps:/proc/swaps:ro \
  ghcr.io/selkies-project/nvidia-egl-desktop:latest
```

Static SM Partitioning

Step 1 — Update MPS daemon with static partitioning:

bash

```
sudo systemctl stop cuda-mps

sudo tee /etc/systemd/system/cuda-mps.service > /dev/null << 'EOF'
[Unit]
Description=CUDA MPS Control Daemon — Static SM Partitioning
After=nvidia-persistenced.service

[Service]
Type=forking
Environment=CUDA_VISIBLE_DEVICES=0
Environment=CUDA_MPS_PIPE_DIRECTORY=/tmp/nvidia-mps
```

```
Environment=CUDA_MPS_LOG_DIRECTORY=/tmp/nvidia-log
ExecStartPre=/usr/bin/nvidia-smi -pm 1
ExecStart=/usr/bin/nvidia-cuda-mps-control -d
ExecStop=/bin/bash -c "echo quit | /usr/bin/nvidia-cuda-mps-control"
Restart=on-failure
RestartSec=10

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now cuda-mps
sudo systemctl status cuda-mps
```

Step 2 — Helper script for orchestrator to calculate SM% from VRAM:

bash

```
sudo tee /usr/local/bin/calc-sm-percent << 'EOF'
#!/bin/bash
# Usage: calc-sm-percent <container_vram_gb> <total_gpu_vram_gb>
# Returns: integer SM% to use for CUDA_MPS_ACTIVE_THREAD_PERCENTAGE
#
# RTX 4090 POC:  total_gpu_vram_gb = 24
# RTX 5090 PROD: total_gpu_vram_gb = 32

CONTAINER_VRAM=${1:?Usage: calc-sm-percent <container_vram_gb> <total_gpu_vram_gb>}
TOTAL_VRAM=${2:?Usage: calc-sm-percent <container_vram_gb> <total_gpu_vram_gb>}

# Integer math: floor(vram/total * 100), minimum 5%
```

```
PCT=$(( (CONTAINER_VRAM * 100) / TOTAL_VRAM ))
[ $PCT -lt 5 ] && PCT=5

echo $PCT
EOF
chmod +x /usr/local/bin/calc-sm-percent

# Verify the math
echo "=== SM% by VRAM tier (RTX 4090, 24GB pool) ==="
for vram in 2 4 6 8 12 16 24; do
  pct=$(calc-sm-percent $vram 24)
  echo "  ${vram}GB VRAM → ${pct}% SMs"
done

echo ""
echo "=== SM% by VRAM tier (RTX 5090, 32GB pool) ==="
for vram in 4 8 16 32; do
  pct=$(calc-sm-percent $vram 32)
  echo "  ${vram}GB VRAM → ${pct}% SMs"
done
```

Step 3 — Stop all containers and relaunch with proportional SM%:

```
bash
docker stop selkies-test-1 selkies-test-2 selkies-test-3 selkies-test-4
docker rm selkies-test-1 selkies-test-2 selkies-test-3 selkies-test-4

# Total VRAM pool on RTX 4090
TOTAL_VRAM_GB=24
```

```
for i in 1 2 3 4; do
  NGINX_P=$((8080 + i))
  SELKIES_P=$((9080 + i))
  METRICS_P=$((19080 + i))
  DISPLAY_N=$((19 + i))
  CPUSET="$(( ($i-1)*4 ))-$(( ($i-1)*4+3 ))"

  # VRAM per tier
  VRAM_GB=$([ $i -le 2 ] && echo "4" || echo "8")
  VRAM="${VRAM_GB}192m"  # in MB for HAMi
  VRAM_MPS="0=${VRAM_GB}G"

  # SM% proportional to VRAM
  SM_PCT=$(calc-sm-percent $VRAM_GB $TOTAL_VRAM_GB)

  SESSION_HOST="ws-$(openssl rand -hex 4)"
  TOKEN=$(openssl rand -hex 16)

  echo "Container $i → VRAM: ${VRAM_GB}GB | SM: ${SM_PCT}% | cpuset: $CPUSET | port:
$NGINX_P | token: $TOKEN"

  docker run -d \
    --name selkies-test-$i \
    --hostname $SESSION_HOST \
    --gpus all \
    --cpus=4 \
    --cpuset-cpus=$CPUSET \
    --memory=8g \
```

```
--ipc=host --network=host --tmpfs /dev/shm:rw \
-e TZ=UTC \
-e DISPLAY=:$DISPLAY_N \
-e DISPLAY_SIZEW=1920 -e DISPLAY_SIZEH=1080 -e DISPLAY_REFRESH=60 \
-e SELKIES_ENCODER=nvh264enc \
-e SELKIES_ENABLE_BASIC_AUTH=true \
-e SELKIES_BASIC_AUTH_PASSWORD=$TOKEN \
-e NGINX_PORT=$NGINX_P \
-e SELKIES_PORT=$SELKIES_P \
-e SELKIES_METRICS_HTTP_PORT=$METRICS_P \
-e PASSWD=mypasswd \
-e CUDA_VISIBLE_DEVICES=0 \
-e CUDA_NVRTC_ARCH=89 \
-e __NV_PRIME_RENDER_OFFLOAD=1 \
-e __GLX_VENDOR_LIBRARY_NAME=nvidia \
-e CUDA_DEVICE_MEMORY_LIMIT_0="${VRAM_GB}192m" \
-e CUDA_DEVICE_SM_LIMIT=$SM_PCT \
-e CUDA_MPS_PINNED_DEVICE_MEM_LIMIT="$VRAM_MPS" \
-e CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=$SM_PCT \
-e CUDA_MPS_ENABLE_PER_CTX_DEVICE_MULTIPROCESSOR_PARTITIONING=1 \
-e SELKIES_TURN_HOST=106.51.247.170 \
-e SELKIES_TURN_PORT=3478 \
-e SELKIES_TURN_USERNAME=selkies \
-e SELKIES_TURN_PASSWORD=wVIAbfwkgkxjaCiZVX4BDsdU \
-e SELKIES_TURN_PROTOCOL=tcp \
-v /mnt/nfs/users/testuser$i:/home/ubuntu \
-v /usr/lib/libvgpu.so:/usr/lib/libvgpu.so \
-v /tmp/vgpulock-$i:/tmp/vgpulock \
-v /usr/bin/nvidia-smi:/usr/bin/nvidia-smi.real \
```

```
    -v /tmp/nvidia-smi-wrapper:/usr/bin/nvidia-smi \
    -v /usr/bin/passwd:/usr/bin/passwd.real \
    -v /tmp/passwd-wrapper:/usr/bin/passwd \
    -v /tmp/supervisord-hami.conf:/etc/supervisord.conf \
    -v /tmp/bash.bashrc.orig:/etc/bash.bashrc \
    -v /var/lib/lxcfs/proc/cpuinfo:/proc/cpuinfo:ro \
    -v /var/lib/lxcfs/proc/meminfo:/proc/meminfo:ro \
    -v /var/lib/lxcfs/proc/stat:/proc/stat:ro \
    -v /var/lib/lxcfs/proc/uptime:/proc/uptime:ro \
    -v /var/lib/lxcfs/proc/loadavg:/proc/loadavg:ro \
    -v /var/lib/lxcfs/proc/diskstats:/proc/diskstats:ro \
    -v /var/lib/lxcfs/proc/swaps:/proc/swaps:ro \
    ghcr.io/selkies-project/nvidia-egl-desktop:latest
done
```

Paste the output of the container launch lines — should show:

```
Container 1 → VRAM: 4GB | SM: 16% | ...
Container 2 → VRAM: 4GB | SM: 16% | ...
Container 3 → VRAM: 8GB | SM: 33% | ...
Container 4 → VRAM: 8GB | SM: 33% | ...
```

```
TOKEN=$(openssl rand -hex 16)
echo "Access: http://100.100.66.101:8083 | token: $TOKEN"
```

Full start!!

```
docker stop selkies-test-1 selkies-test-2 selkies-test-3 selkies-test-4
docker rm selkies-test-1 selkies-test-2 selkies-test-3 selkies-test-4

TOTAL_VRAM_GB=24

for i in 1 2 3 4; do
  NGINX_P=$((8080 + i))
  SELKIES_P=$((9080 + i))
  METRICS_P=$((19080 + i))
  DISPLAY_N=$((19 + i))
  CPUSET="$(( ($i-1)*4 ))-$(( ($i-1)*4+3 ))"
  VRAM_GB=$([ $i -le 2 ] && echo "4" || echo "8")
  SM_PCT=$(calc-sm-percent $VRAM_GB $TOTAL_VRAM_GB)
  SESSION_HOST="ws-$(openssl rand -hex 4)"
  TOKEN=$(openssl rand -hex 16)
  MEM_BYTES=$((8*1024*1024*1024))
  echo "Container $i → VRAM: ${VRAM_GB}GB | SM: ${SM_PCT}% | cpuset: $CPUSET | port: $NGINX_P | token: $TOKEN"

  docker run -d \
    --name selkies-test-$i \
    --hostname $SESSION_HOST \
    --gpus all \
    --cpus=4 \
    --cpuset-cpus=$CPUSET \
```

```
--memory=8g \
--ipc=host --network=host --tmpfs /dev/shm:rw \
-e TZ=UTC \
-e DISPLAY=:$DISPLAY_N \
-e DISPLAY_SIZEW=1920 -e DISPLAY_SIZEH=1080 -e DISPLAY_REFRESH=60 \
-e SELKIES_ENCODER=nvh264enc \
-e SELKIES_ENABLE_BASIC_AUTH=true \
-e SELKIES_BASIC_AUTH_PASSWORD=$TOKEN \
-e NGINX_PORT=$NGINX_P \
-e SELKIES_PORT=$SELKIES_P \
-e SELKIES_METRICS_HTTP_PORT=$METRICS_P \
-e PASSWD=mypasswd \
-e CUDA_VISIBLE_DEVICES=0 \
-e CUDA_NVRTC_ARCH=89 \
-e __NV_PRIME_RENDER_OFFLOAD=1 \
-e __GLX_VENDOR_LIBRARY_NAME=nvidia \
-e CUDA_DEVICE_MEMORY_LIMIT_0="${VRAM_GB}192m" \
-e CUDA_DEVICE_SM_LIMIT=$SM_PCT \
-e CUDA_MPS_PINNED_DEVICE_MEM_LIMIT="0=${VRAM_GB}G" \
-e CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=$SM_PCT \
-e CUDA_MPS_ENABLE_PER_CTX_DEVICE_MULTIPROCESSOR_PARTITIONING=1 \
-e CONTAINER_MEMORY_LIMIT_BYTES=$MEM_BYTES \
-e SELKIES_TURN_HOST=106.51.247.170 \
-e SELKIES_TURN_PORT=3478 \
-e SELKIES_TURN_USERNAME=selkies \
-e SELKIES_TURN_PASSWORD=wVIAbfwkgkxjaCiZVX4BDsdU \
-e SELKIES_TURN_PROTOCOL=tcp \
-v /mnt/nfs/users/testuser$i:/home/ubuntu \
-v /usr/lib/libvgpu.so:/usr/lib/libvgpu.so \
```

```
    -v /usr/lib/fake_sysconf.so:/usr/lib/fake_sysconf.so \
    -v /tmp/vgpulock-$i:/tmp/vgpulock \
    -v /usr/bin/nvidia-smi:/usr/bin/nvidia-smi.real \
    -v /tmp/nvidia-smi-wrapper:/usr/bin/nvidia-smi \
    -v /usr/bin/passwd:/usr/bin/passwd.real \
    -v /tmp/passwd-wrapper:/usr/bin/passwd \
    -v /tmp/supervisord-hami.conf:/etc/supervisord.conf \
    -v /tmp/bash.bashrc.orig:/etc/bash.bashrc \
    -v /var/lib/lxcfs/proc/cpuinfo:/proc/cpuinfo:ro \
    -v /var/lib/lxcfs/proc/meminfo:/proc/meminfo:ro \
    -v /var/lib/lxcfs/proc/stat:/proc/stat:ro \
    -v /var/lib/lxcfs/proc/uptime:/proc/uptime:ro \
    -v /var/lib/lxcfs/proc/loadavg:/proc/loadavg:ro \
    -v /var/lib/lxcfs/proc/diskstats:/proc/diskstats:ro \
    -v /var/lib/lxcfs/proc/swaps:/proc/swaps:ro \
    -v /tmp/container-$i-cpu:/sys/devices/system/cpu:ro \
    ghcr.io/selkies-project/nvidia-egl-desktop:latest
done




docker run -d \
  --name selkies-test-2 \
  --hostname ws-$(openssl rand -hex 4) \
  --gpus all \
  --cpus=4 \
  --cpuset-cpus=0-3 \
  --memory=8g \
```

```
--ipc=host --network=host --tmpfs /dev/shm:rw \
-e TZ=UTC \
-e DISPLAY=:21 \
-e DISPLAY_SIZEW=1920 -e DISPLAY_SIZEH=1080 -e DISPLAY_REFRESH=60 \
-e SELKIES_ENCODER=nvh264enc \
-e SELKIES_ENABLE_BASIC_AUTH=true \
-e SELKIES_BASIC_AUTH_PASSWORD=123 \
-e NGINX_PORT=8082 \
-e SELKIES_PORT=9082 \
-e SELKIES_METRICS_HTTP_PORT=19082 \
-e PASSWD=mypasswd \
-e CUDA_VISIBLE_DEVICES=0 \
-e CUDA_NVRTC_ARCH=89 \
-e __NV_PRIME_RENDER_OFFLOAD=1 \
-e __GLX_VENDOR_LIBRARY_NAME=nvidia \
-e CUDA_DEVICE_MEMORY_LIMIT_0=4096m \
-e CUDA_DEVICE_SM_LIMIT=16 \
-e CUDA_MPS_PINNED_DEVICE_MEM_LIMIT="0=4G" \
-e CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=16 \
-e CUDA_MPS_ENABLE_PER_CTX_DEVICE_MULTIPROCESSOR_PARTITIONING=1 \
-e CONTAINER_MEMORY_LIMIT_BYTES=$((8*1024*1024*1024)) \
-e SELKIES_TURN_HOST=106.51.247.170 \
-e SELKIES_TURN_PORT=3478 \
-e SELKIES_TURN_USERNAME=selkies \
-e SELKIES_TURN_PASSWORD=wVIAbfwkgkxjaCiZVX4BDsdU \
-e SELKIES_TURN_PROTOCOL=tcp \
-v /mnt/nfs/users/testuser2:/home/ubuntu \
-v /usr/lib/libvgpu.so:/usr/lib/libvgpu.so \
-v /usr/lib/fake_sysconf.so:/usr/lib/fake_sysconf.so \
```

```
  -v /tmp/vgpulock-2:/tmp/vgpulock \
  -v /usr/bin/nvidia-smi:/usr/bin/nvidia-smi.real \
  -v /tmp/nvidia-smi-wrapper:/usr/bin/nvidia-smi \
  -v /usr/bin/passwd:/usr/bin/passwd.real \
  -v /tmp/passwd-wrapper:/usr/bin/passwd \
  -v /tmp/supervisord-hami.conf:/etc/supervisord.conf \
  -v /tmp/bash.bashrc.orig:/etc/bash.bashrc \
  -v /var/lib/lxcfs/proc/cpuinfo:/proc/cpuinfo:ro \
  -v /var/lib/lxcfs/proc/meminfo:/proc/meminfo:ro \
  -v /var/lib/lxcfs/proc/stat:/proc/stat:ro \
  -v /var/lib/lxcfs/proc/uptime:/proc/uptime:ro \
  -v /var/lib/lxcfs/proc/loadavg:/proc/loadavg:ro \
  -v /var/lib/lxcfs/proc/diskstats:/proc/diskstats:ro \
  -v /var/lib/lxcfs/proc/swaps:/proc/swaps:ro \
  -v /tmp/container-2-cpu:/sys/devices/system/cpu:ro \
  --label laas.session_id=$(openssl rand -hex 8) \
  --label laas.user_id=testuser2@laas.in \
  --label laas.tier=pro \
  --label laas.session_type=stateful \
  --label laas.node=node-mvp \
  ghcr.io/selkies-project/nvidia-egl-desktop:latest
```

Correct Docker command: (prior nvidia-smi type -> C)
To use MPS

```
-v /tmp/nvidia-mps:/tmp/nvidia-mps \
-v /tmp/nvidia-log:/tmp/nvidia-log \

docker run -d \
  --name selkies-test-2 \
```

```
--hostname ws-$(openssl rand -hex 4) \
--gpus all \
--cpus=4 \
--cpuset-cpus=0-3 \
--memory=8g \
--ipc=host --network=host --tmpfs /dev/shm:rw \
-e TZ=UTC \
-e DISPLAY=:21 \
-e DISPLAY_SIZEW=1920 -e DISPLAY_SIZEH=1080 -e DISPLAY_REFRESH=60 \
-e SELKIES_ENCODER=nvh264enc \
-e SELKIES_ENABLE_BASIC_AUTH=true \
-e SELKIES_BASIC_AUTH_PASSWORD=123 \
-e NGINX_PORT=8082 \
-e SELKIES_PORT=9082 \
-e SELKIES_METRICS_HTTP_PORT=19082 \
-e PASSWD=mypasswd \
-e CUDA_VISIBLE_DEVICES=0 \
-e CUDA_NVRTC_ARCH=89 \
-e __NV_PRIME_RENDER_OFFLOAD=1 \
-e __GLX_VENDOR_LIBRARY_NAME=nvidia \
-e CUDA_DEVICE_MEMORY_LIMIT_0=4096m \
-e CUDA_DEVICE_SM_LIMIT=16 \
-e CUDA_MPS_PINNED_DEVICE_MEM_LIMIT="0=4G" \
-e CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=16 \
-e CUDA_MPS_ENABLE_PER_CTX_DEVICE_MULTIPROCESSOR_PARTITIONING=1 \
-e CONTAINER_MEMORY_LIMIT_BYTES=$((8*1024*1024*1024)) \
-e SELKIES_TURN_HOST=106.51.247.170 \
-e SELKIES_TURN_PORT=3478 \
-e SELKIES_TURN_USERNAME=selkies \
```

```
-e SELKIES_TURN_PASSWORD=wVIAbfwkgkxjaCiZVX4BDsdU \
-e SELKIES_TURN_PROTOCOL=tcp \
-v /tmp/nvidia-mps:/tmp/nvidia-mps \
-v /tmp/nvidia-log:/tmp/nvidia-log \
-v /mnt/nfs/users/testuser2:/home/ubuntu \
-v /usr/lib/libvgpu.so:/usr/lib/libvgpu.so \
-v /usr/lib/fake_sysconf.so:/usr/lib/fake_sysconf.so \
-v /tmp/vgpulock-2:/tmp/vgpulock \
-v /usr/bin/nvidia-smi:/usr/bin/nvidia-smi.real \
-v /tmp/nvidia-smi-wrapper:/usr/bin/nvidia-smi \
-v /usr/bin/passwd:/usr/bin/passwd.real \
-v /tmp/passwd-wrapper:/usr/bin/passwd \
-v /tmp/supervisord-hami.conf:/etc/supervisord.conf \
-v /tmp/bash.bashrc.orig:/etc/bash.bashrc \
-v /var/lib/lxcfs/proc/cpuinfo:/proc/cpuinfo:ro \
-v /var/lib/lxcfs/proc/meminfo:/proc/meminfo:ro \
-v /var/lib/lxcfs/proc/stat:/proc/stat:ro \
-v /var/lib/lxcfs/proc/uptime:/proc/uptime:ro \
-v /var/lib/lxcfs/proc/loadavg:/proc/loadavg:ro \
-v /var/lib/lxcfs/proc/diskstats:/proc/diskstats:ro \
-v /var/lib/lxcfs/proc/swaps:/proc/swaps:ro \
-v /tmp/container-2-cpu:/sys/devices/system/cpu:ro \
--label laas.session_id=$(openssl rand -hex 8) \
--label laas.user_id=testuser2@laas.in \
--label laas.tier=pro \
--label laas.session_type=stateful \
--label laas.node=node-mvp \
ghcr.io/selkies-project/nvidia-egl-desktop:latest
```

cAdvisor is set on the host!
docker run -d \

  --name cadvisor \

  --privileged \

  -p 8999:8080 \

  -v /:/rootfs:ro \

  -v /var/run:/var/run:rw \

  -v /var/lib/docker:/var/lib/docker:ro \

  gcr.io/cadvisor/cadvisor:v0.52.1 \

  -housekeeping_interval=15s

Restart Policy

```bash
docker run -d \
  --name selkies-test-1 \
  --hostname ws-$(openssl rand -hex 4) \
  --restart unless-stopped \
  --gpus all \
```

important!!!

CoreDump due to conflict between bash.rc and Hami!
You removed bash.bashrc mount → LD_PRELOAD never gets set → HAMi never loads → full 23.5GB visible.

The segfault was because your /tmp/bash.bashrc.orig was generated from the **old cached image**. You pulled a new image — the bashrc format changed slightly. Regenerate it from the new image:

```
# Regenerate bash.bashrc from the current image
docker run --rm --entrypoint cat \
  ghcr.io/selkies-project/nvidia-egl-desktop:latest \
  /etc/bash.bashrc > /tmp/bash.bashrc.orig

# Append HAMi + fake_sysconf injection
cat >> /tmp/bash.bashrc.orig << 'EOF'

# LaaS resource interceptors
if [ "${HAMI_INJECTED}" != "1" ]; then
  export LD_PRELOAD="/usr/lib/libvgpu.so /usr/lib/fake_sysconf.so"
  export HAMI_INJECTED=1
  export SYSCONF_INJECTED=1
  mkdir -p /tmp/vgpulock 2>/dev/null
fi
EOF

chmod 644 /tmp/bash.bashrc.orig

# Verify injection is in there
tail -10 /tmp/bash.bashrc.orig
```

stop and relaunch!!

What went wrong when I decided to delete user 1 and user 2 to expand 2 and 3’s disk alloc

sudo zpool create -f datapool /vg_containers/nas_pool_30g.img

zpool create with -f on a file that already has a pool **wipes it and creates a fresh empty pool**. The cp in step 4 copied the data correctly, but then zpool create destroyed it immediately after.

The correct command should have been zpool import, not zpool create.